# Práctica 4: Construye tu propio microprocesador

### Introducción

En la segunda práctica simulamos circuitos digitales. En la tercera montamos un circuito y programamos un microprocesador real. Ahora damos el último paso: vamos a construir nuestro propio microprocesador y a programarlo.

El procesador será un RISC-V monociclo que ejecuta el juego de instrucciones RV32I, salvo las instrucciones de sistema. Lo construiremos y probaremos en **Digital** y, una vez funcione, podremos llevarlo a una FPGA y verlo ejecutar programas en hardware real.

No partimos de cero. Las piezas más largas de escribir, como la ALU o la unidad de control, se entregan ya hechas y probadas, así que el trabajo consiste en diseñar el **camino de datos**: decidir qué recorrido sigue cada instrucción y conectar los bloques para que lo haga.

### Material que se entrega

| Material | Contenido |
|---|---|
| `Controller` | Unidad de control: decodificador (`controldec`) y lógica de salto (`branchlogic`) |
| `alu` | Unidad aritmético-lógica |
| `extend` | Extensor de inmediatos para todos los formatos de instrucción |
| `storealign` | Coloca el dato y genera la máscara de bytes en `sb`, `sh` y `sw` |
| `loadext` | Extrae el byte o la media palabra y extiende el signo en las cargas |
| Circuito de partida | Reloj, reset, memoria de instrucciones con el programa de prueba y los bloques anteriores colocados, sin cablear |
| `prueba_completa.hex` | Programa de prueba que se comprueba a sí mismo. Habrá que cargarlo en la memoria de instrucciones con el path que correspondan en tu ordenador portátil. |
| `prueba_completa_listado.txt` | Listado del programa y tabla de pruebas |
| Esquema de bloques | Especificación del procesador que hay que construir |

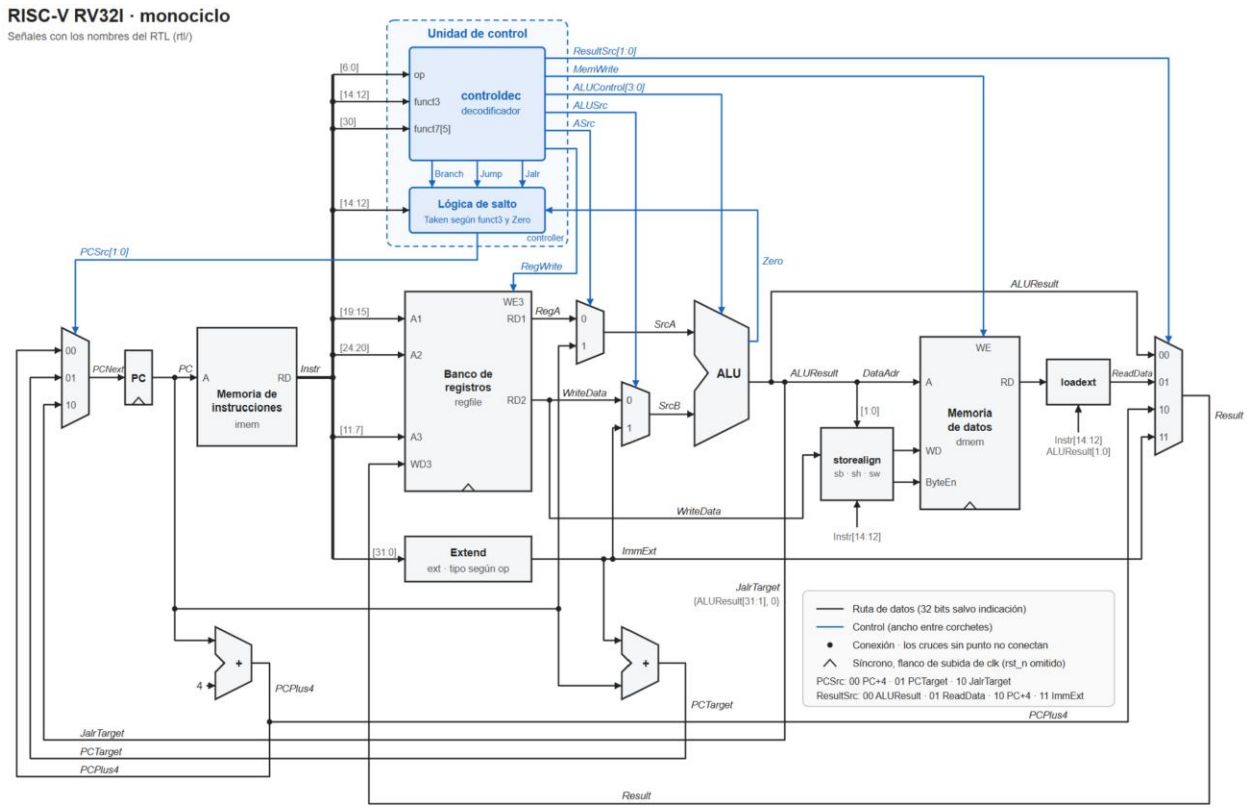
Cada bloque entregado tiene su propio caso de prueba y funciona correctamente. Si algo falla, el problema está en las conexiones.

### Especificación

El procesador ejecuta 37 instrucciones de RV32I en un solo ciclo de reloj cada una. Quedan fuera `fence`, `ecall`, `ebreak` y las instrucciones de registros de control.

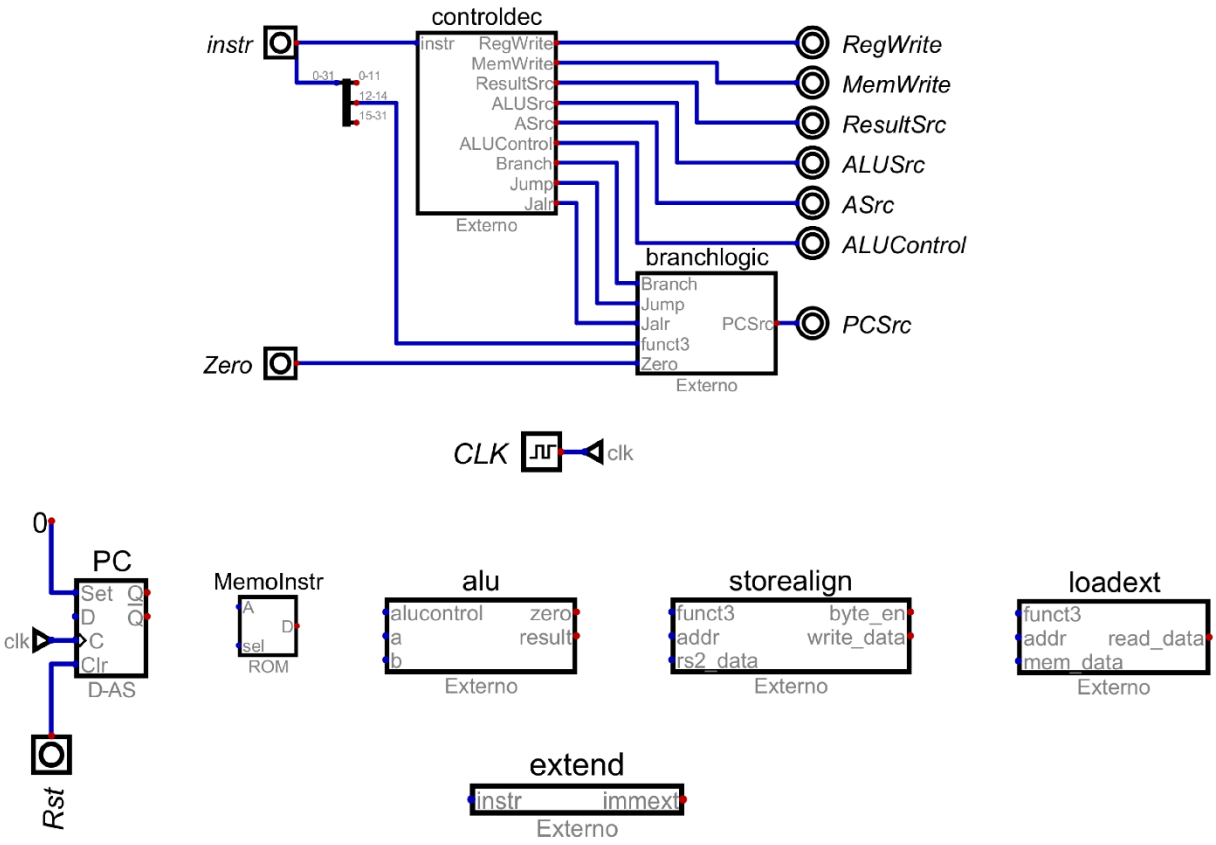
| Tipo | Instrucciones |
|---|---|
| R | `add`, `sub`, `sll`, `slt`, `sltu`, `xor`, `srl`, `sra`, `or`, `and` |
| I aritméticas | `addi`, `slti`, `sltiu`, `xori`, `ori`, `andi`, `slli`, `srli`, `srai` |
| I cargas | `lb`, `lh`, `lw`, `lbu`, `lhu` |
| S | `sb`, `sh`, `sw` |
| B | `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu` |
| U | `lui`, `auipc` |
| J e I salto | `jal`, `jalr` |

## Esquema de bloques




El esquema indica qué bloques hay y cómo se conectan, pero no cómo se construyen en Digital. Ese es tu trabajo.

De ese esquema ya tienes en Digital lo siguiente:




## Campos de la instrucción

| Campo | Bits | Uso |
|---|---|---|
| `opcode` | 6:0 | Tipo de instrucción (lo usa la unidad de control) |
| `rd` | 11:7 | Registro destino |
| `funct3` | 14:12 | Variante de la operación |
| `rs1` | 19:15 | Primer registro fuente |
| `rs2` | 24:20 | Segundo registro fuente |
| `funct7` | 31:25 | Distingue `sub`/`add` y `sra`/`srl` (bit 30) |

En los formatos I, S, B, U y J, los bits que no son registros forman el inmediato. No hace falta separarlo: `extend` recibe la instrucción entera y devuelve el inmediato ya extendido a 32 bits.

## Memoria

- **Memoria de instrucciones:** ROM de 1024 palabras de 32 bits (4 KB), en la dirección 0.
- **Memoria de datos:** 4 KB, direccionable por bytes, en la dirección 0. Se construye con cuatro RAM de 1024 × 8 bits, una por cada byte de la palabra.
- **Registro `x0`:** vale siempre 0, aunque una instrucción intente escribir en él.

### Interfaces de los bloques entregados

Todos los bloques son combinacionales: sus salidas cambian en cuanto cambian sus entradas, sin esperar al reloj.

| Bloque | Entradas (bits) | Salidas (bits) |
|---|---|---|
| `Controller` | `instr` (32), `Zero` (1) | `RegWrite` (1), `MemWrite` (1), `ResultSrc` (2), `ALUSrc` (1), `ASrc` (1), `ALUControl` (4), `PCSrc` (2) |
| `alu` | `alucontrol` (4), `a` (32), `b` (32) | `result` (32), `zero` (1) |
| `extend` | `instr` (32) | `immext` (32) |
| `storealign` | `funct3` (3), `addr` (2), `rs2_data` (32) | `write_data` (32), `byte_en` (4) |
| `loadext` | `funct3` (3), `addr` (2), `mem_data` (32) | `read_data` (32) |

Algunas aclaraciones:

- **`zero`** vale 1 cuando el resultado de la ALU es 0. En los saltos condicionales, la ALU compara `rs1` y `rs2`, y `Controller` usa `Zero` para decidir si se salta.
- **`addr`** de `storealign` y `loadext` son los 2 bits menos significativos de la dirección: indican qué byte de la palabra se usa.
- **`write_data`** es el dato colocado en su posición dentro de la palabra. **`byte_en`** tiene un bit por byte: el bit *i* a 1 indica que se escribe el byte *i*.
- **`mem_data`** es la palabra completa leída de memoria; `loadext` devuelve en `read_data` el valor que debe ir al registro.

### Codificación de las señales de control

El orden de las entradas de cada multiplexor debe coincidir con esta tabla. Si se conectan en otro orden, el procesador compila y simula, pero elige el dato equivocado.

| Señal | Valor | Selecciona |
|---|---|---|
| `PCSrc` | 00 | PC + 4 |
| | 01 | PC + inmediato (saltos condicionales tomados y `jal`) |
| | 10 | Resultado de la ALU (`jalr`) |
| | 11 | No se usa |
| `ResultSrc` | 00 | Resultado de la ALU |
| | 01 | Dato leído de memoria (`read_data`) |
| | 10 | PC + 4 (`jal` y `jalr`) |
| | 11 | Inmediato extendido (`lui`) |

| Señal | Valor | Selecciona |
|---|---|---|
| `ASrc` | 0 | Registro `rs1` |
| | 1 | PC (`auipc`) |
| `ALUSrc` | 0 | Registro `rs2` |
| | 1 | Inmediato extendido |
| `RegWrite` | 1 | Escribe el resultado en `rd` en el flanco de subida |
| `MemWrite` | 1 | Escribe en memoria en el flanco de subida |

### Operaciones de la ALU

| `ALUControl` | Operación |
|---|---|
| 0000 | Suma |
| 0001 | Resta |
| 0010 | AND |
| 0011 | OR |
| 0101 | Menor que, con signo (`slt`) |
| 1000 | XOR |
| 1001 | Desplazamiento lógico a la izquierda |
| 1010 | Desplazamiento lógico a la derecha |
| 1011 | Desplazamiento aritmético a la derecha |
| 1100 | Menor que, sin signo (`sltu`) |

Esta tabla os sirve para interpretar lo que veis en la simulación: `ALUControl` lo genera `Controller`, no hay que calcularlo.

### Pasos de construcción

Construye el procesador por partes y comprueba cada una antes de seguir. Para comprobar, pon el reloj en modo manual y mira los valores de los cables pasando el ratón por encima.

1. **Camino del PC.** Registro del PC (valor inicial 0), sumador PC + 4 y multiplexor de `PCSrc`. Conectad de momento solo la entrada PC + 4.
    - *Comprobación:* en cada flanco de subida, el PC avanza 4.
2. **Memoria de instrucciones.** La ROM trabaja por palabras y el PC cuenta bytes. Pensad qué bits del PC hay que llevar a su dirección.
    - *Comprobación:* la instrucción leída coincide con el listado para cada valor del PC.

3. **Decodificación y banco de registros.** Separa los campos de la instrucción y conecta el banco de registros (32 registros de 32 bits). Añade la lógica que impide escribir en `x0`.
4. **Unidad de control.** Conecta la entrada de `Controller` y llevad sus señales hacia donde se usarán.
5. **ALU y operandos.** Multiplexores de `ASrc` y `ALUSrc`, extensor de inmediatos y ALU. Lleva provisionalmente el resultado de la ALU a la entrada de escritura del banco de registros.
    - *Comprobación:* tras el primer flanco, `x1` = 5; tras el segundo, `x29` = 5.
6. **Memoria de datos.** Cuatro RAM de 1024 × 8 bits, `storealign` y `loadext`. Piensa qué bits de la dirección van a las RAM, cuáles a `storealign` y `loadext`, y cuándo debe escribir cada RAM.
7. **Multiplexor de resultado.** Sus cuatro entradas, en el orden de `ResultSrc`.
8. **Saltos.** Sumador PC + inmediato y las entradas restantes del multiplexor del PC.
9. **Prueba completa.** Ejecuta el programa de prueba hasta el final (siguiente apartado).

## Prueba y criterio de éxito

El procesador funciona si, al ejecutar el programa de prueba completo, termina con `x27` = 0x0000600D y `x30` = 0.

El programa ejecuta 65 pruebas. Cada una ejecuta una instrucción, compara el resultado con el valor esperado y anota los fallos. Al terminar, el PC se queda fijo en 0x4F4, en un bucle infinito.

| Registro | Valor correcto | Significado |
|---|---|---|
| `x27` | 0x0000600D | El programa ha llegado al final |
| `x30` | 0 | Número de pruebas que han fallado |
| `x31` | 0 | Número de la última prueba fallida |

Los registros `x29`, `x30` y `x31` están reservados para las comprobaciones.

## Si algo falla

1. Busca el número de `x31` en la tabla de pruebas de `prueba_completa_listado.txt`. Indica qué instrucción falla, el valor esperado y su dirección.
2. Reinicia la simulación y avanzad ciclo a ciclo hasta esa dirección.
3. Antes del flanco de reloj de la instrucción que falla, comprueba las señales de control, los operandos de la ALU y el valor que llega al banco de registros o a la memoria. El primer valor que no cuadre indica dónde está el error.

Si `x27` no vale 0x600D, el programa no ha llegado al final: probablemente un salto lleva a una dirección equivocada. Mirad el valor del PC.

## Errores típicos

Casi todos los problemas de esta práctica se deben a la configuración de los componentes de Digital, no a la lógica del procesador.

| Síntoma | Causa probable |
|---|---|
| «Se necesitan *n* bits, pero se han encontrado *m*» | Un componente con la anchura equivocada. Revisad *Bits de datos* de puertas, multiplexores, constantes y tierras (una tierra es de 1 bit por defecto), y *Bits de dirección* del banco de registros y las memorias. |
| «No pudo crearse el proceso externo» | Un bloque Externo con el tipo de aplicación en *Genérico* en lugar de *IVerilog*, o el archivo sin guardar después de cambiarlo. |
| El PC no empieza en 0 | El *Valor por defecto* del registro del PC no es 0. |
| El procesador ejecuta una de cada cuatro instrucciones | La ROM recibe la dirección en bytes en lugar de en palabras. |
| `lw` y `sw` de direcciones consecutivas acceden a posiciones salteadas | Lo mismo en la memoria de datos. |
| La ROM o la RAM no sacan ningún dato | La entrada `sel` de la ROM o `ld` de las RAM no está a 1. |
| `x0` cambia de valor | Falta la lógica que impide escribir en `x0`. |
| Los registros conservan valores de una ejecución anterior | `Rst` solo pone a 0 el PC. Para empezar de cero hay que detener la simulación y volver a iniciarla. |
| El PC no avanza al hacer clic en el reloj | La simulación está en modo puerta a puerta, o el reloj está en tiempo real. Usad el modo normal y el reloj manual. |
| Una instrucción funciona y otra parecida no | Entradas de un multiplexor en orden distinto al de la tabla de codificación. |

## Entrega

Demostración, en clase, del funcionamiento con algún programa sencillo que se te ocurra, lo cargas y demuestras que tu microprocesador funciona.

Si antes de que finalice el curso, disponemos de un grabador de FPGA, el programa se podría ejecutar sobre hardware. Una posibilidad es la de reutilizar el programa de la primera práctica (sólo habría que cambiar las direcciones de los MMIO) viendo como funciona el mismo programa, pero ahora sobre hardware real con teclado y display reales controlados por tu propio microprocesador.

Hay que subir a la plataforma una breve descripción (máximo una página) del trabajo realizado.